# INTRODUCTION TO DEEP LEARNING

Course number: 00240332

# Lecture 4: Multi-layer Perceptron

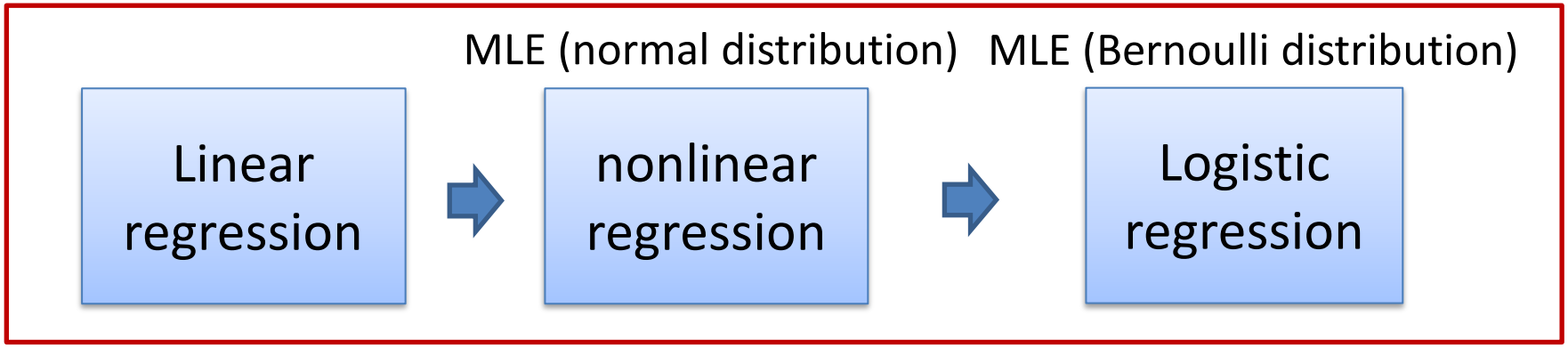
Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

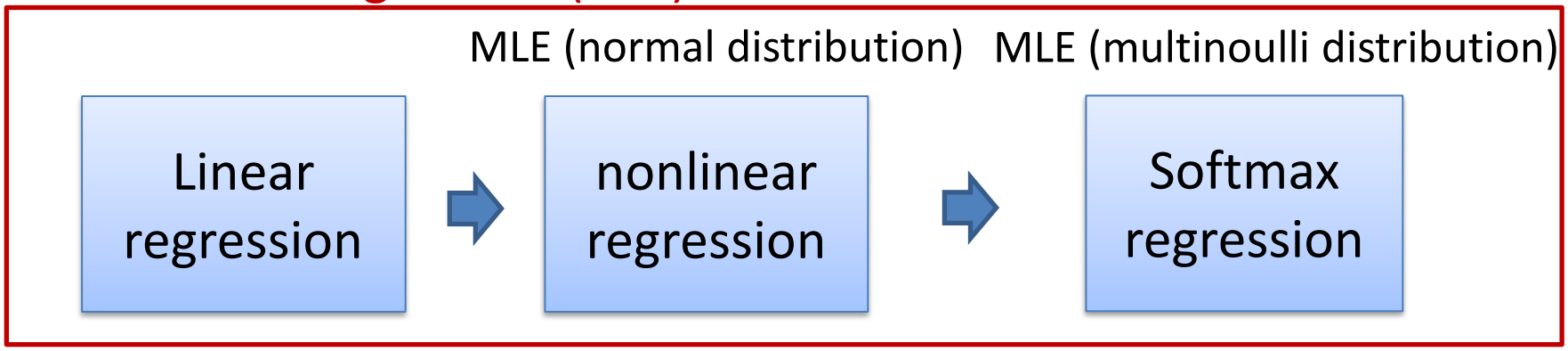
Tsinghua University

# *Last lecture review*

## I. Logistic regression (K=2)




## II. Softmax regression (K>2)



# History of deep learning

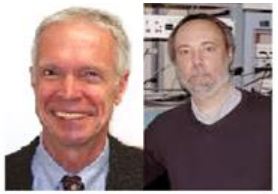
- McCulloch and Pitts neuron: Logic unit
- Perceptron: Binary classification
- Adaline: Widrow-Hoff rule


Pitts McCulloch


Rosenblatt


Widrow


Minsky Papert


Hopfield Tank


Werbos


Rumelhart

SVM

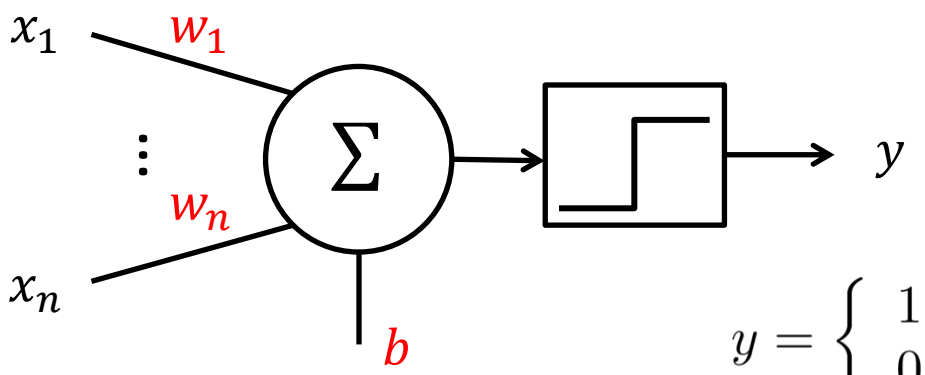

Hinton


LeCun


Bengio

1943 ~1960 1969 1974 1982 1986 1990s 2006

# Recap: Perceptron




$$y = \begin{cases} 1 & \boldsymbol{w}^\top \boldsymbol{x} + b > 0 \\ 0 & \text{else} \end{cases}$$

- For each data points $\boldsymbol{x}^{(j)} \in R^m$ and the corresponding labels $t^{(j)}$
  - Calculate the actual output $y^{(j)}$
  - Update the weights: $\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} + \eta\left(t^{(j)} - y^{(j)}\right)\boldsymbol{x}^{(j)}$;

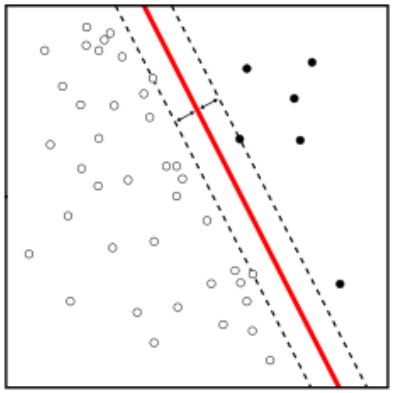
$$b^{\text{new}} = b^{\text{old}} + \eta\left(t^{(j)} - y^{(j)}\right)$$

  where $\eta > 0$ is the learning rate

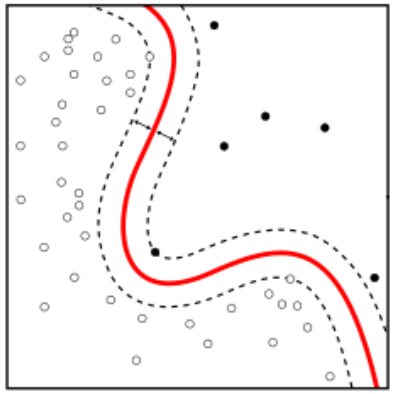
Vector addition

# Limitation

- Perceptron can only solve linearly separable classification problems



linearly separable

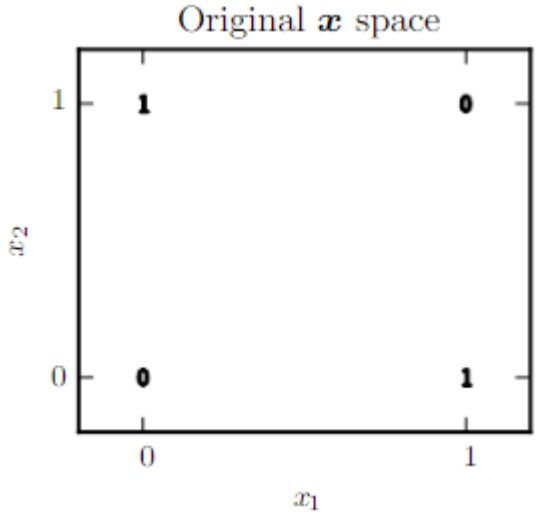


linearly non-separable

How do we deal with linearly non-separable problems?

XOR problem

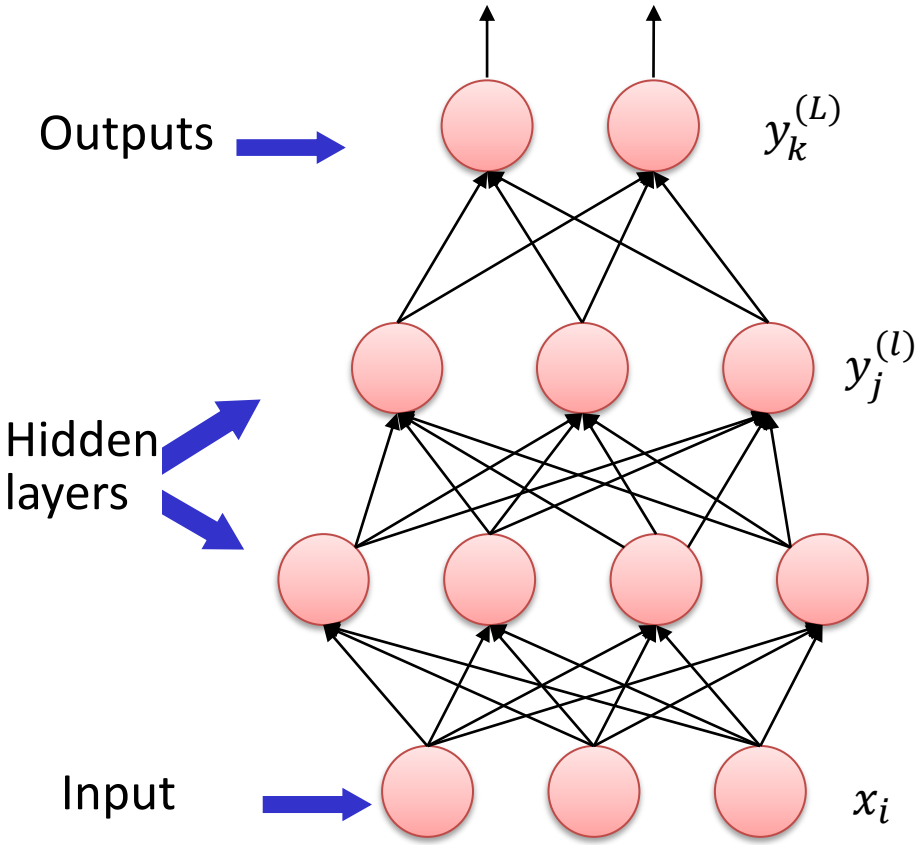
| $x_1$ | $x_2$ | $t$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



# Outline

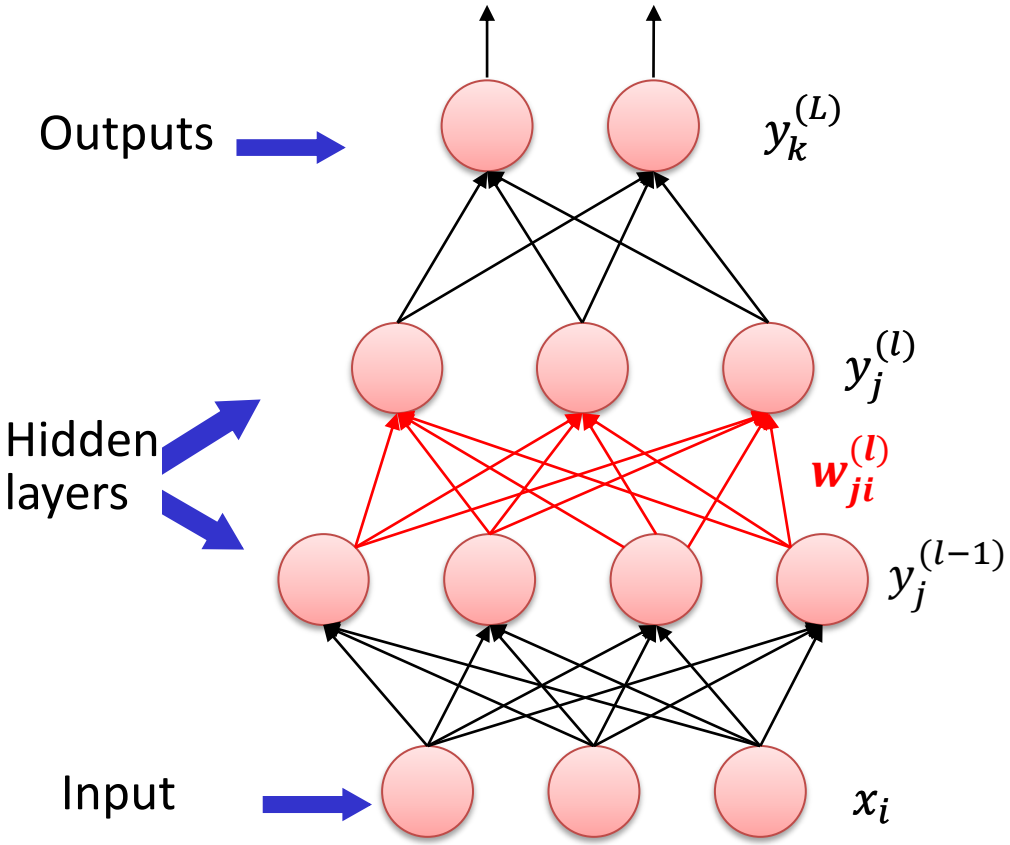
I. Multi-layer perceptron
II. Layer decomposition
III. Training techniques-I
IV. Summary

# Structure of MLP




- There are a total of $L$ layers except the input
- Connections:
  - Full connections between layers
  - No feedback connections between layers
  - No lateral connections in the same layer
- Every neuron receives input from previous layer and fire according to an activation function

# Forward pass




For clarity we don't separate the linear transformation and activation function here

- For $l = 1, \dots, L-1$ calculate the input to neuron $j$ in the $l$-th layer

$$u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$$

and its output

$$y_j^{(l)} = f(u_j^{(l)})$$

where $f(\cdot)$ is activation function

  - Note $y^{(0)} = x$
- $l = L$ corresponds to the output layer
  - The previous lecture

# Activation functions

- Logistic function

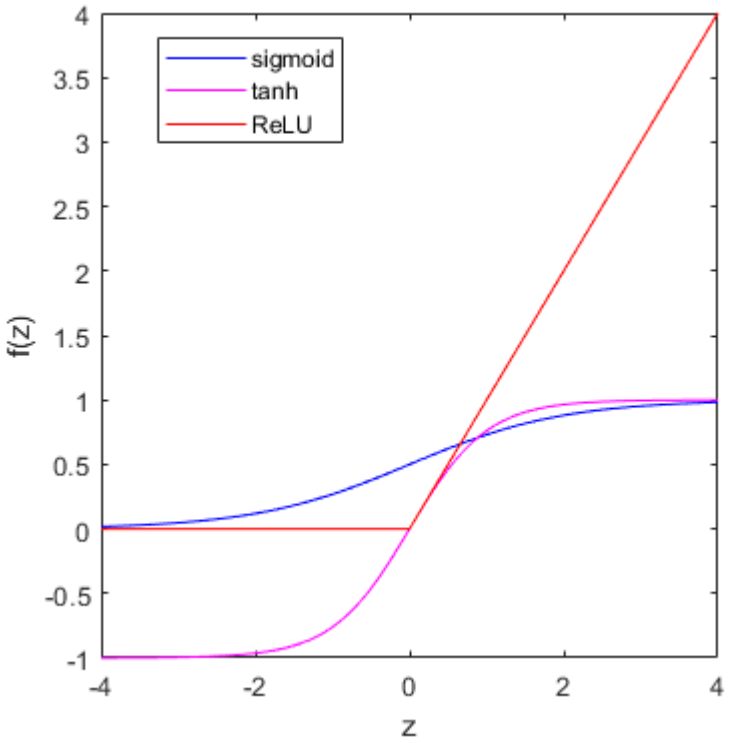
$$f(z) = \frac{1}{1 + \exp(-z)}$$

- Hyperbolic tangent function

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Rectified linear activation function (ReLU)

$$f(z) = \max(0, z)$$



# Activation functions

- Logistic function

$$f(z) = \frac{1}{1 + \exp(-z)}$$

gradient

$$f'(z) = f(z)(1 - f(z))$$

- Hyperbolic tangent function

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

gradient

$$f'(z) = 1 - f(z)^2$$

- Rectified linear activation function (ReLU)

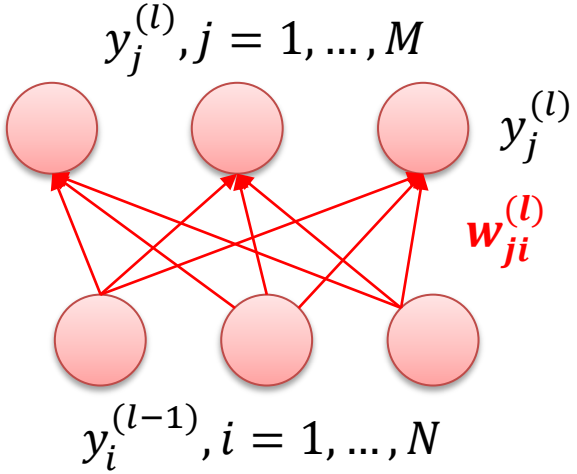
$$f(z) = \max(0, z)$$

gradient

$$f'(z) = \begin{cases} 1, & \text{if } z \geq 0, \\ 0, & \text{else} \end{cases}$$

# Forward pass in the vector-matrix form

- If the previous layer has $N$ neurons and the current layer has $M$ neurons, define the weight matrix and bias vector as




$$\boldsymbol{W}^{(l)} = \begin{pmatrix} w_{11} & \cdots & w_{1N} \\ \vdots & \vdots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{pmatrix} \quad \boldsymbol{b}^{(l)} = \begin{pmatrix} b_1 \\ \vdots \\ b_M \end{pmatrix}$$

- Then for $l = 1, \dots, L-1$

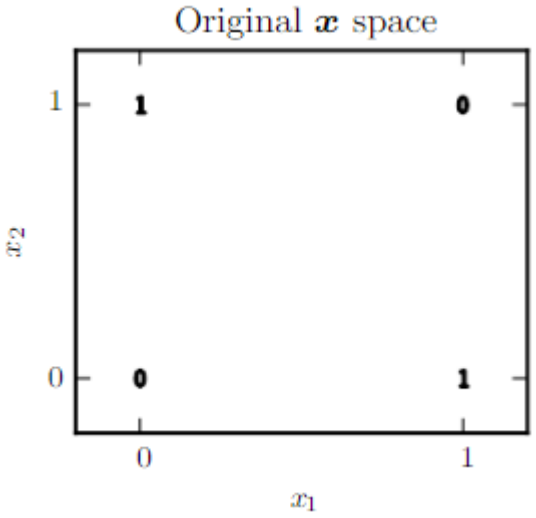
$$\boldsymbol{u}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)} + \boldsymbol{b}^{(l)} \text{ and } \boldsymbol{y}^{(l)} = \boldsymbol{f}(\boldsymbol{u}^{(l)})$$

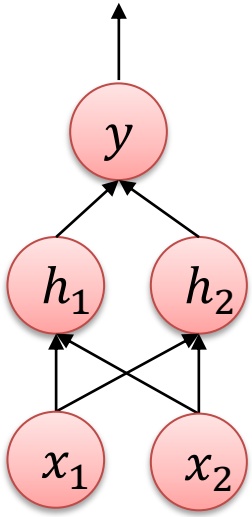
where $\boldsymbol{W}^{(l)} \in R^{M \times N}, \boldsymbol{b}^{(l)} \in R^M, \boldsymbol{u}^{(l)}, \boldsymbol{y}^{(l)} \in R^M, \boldsymbol{y}^{(l-1)} \in R^N$

# XOR problem revisited

- Boolean function:

| $x_1$ | $x_2$ | $t$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |




$$y = \mathrm{ReLU}(\boldsymbol{w}^\top \boldsymbol{h} + b)$$

$$\boldsymbol{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{V}\boldsymbol{x} + \boldsymbol{c})$$

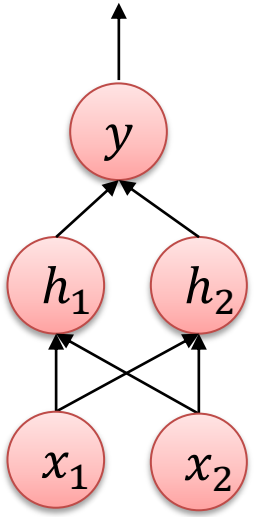
$$\boldsymbol{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \boldsymbol{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

What are $\boldsymbol{h}^{(n)}$ ?

# XOR problem revisited

- Boolean function:

| $x_1$ | $x_2$ | $t$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

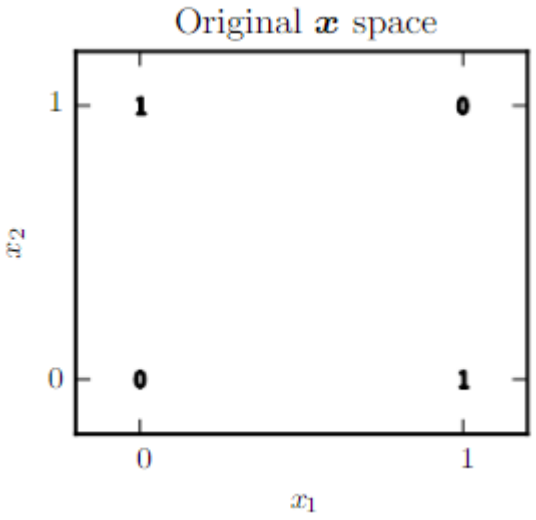


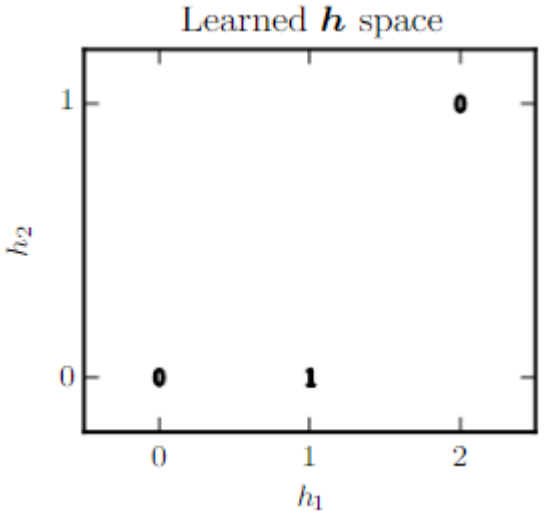

$$y = \text{ReLU}(\boldsymbol{w}^\top \boldsymbol{h} + b)$$

$$\boldsymbol{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\boldsymbol{h} = \text{ReLU}(\boldsymbol{V}\boldsymbol{x} + \boldsymbol{c})$$

$$\boldsymbol{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \boldsymbol{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$



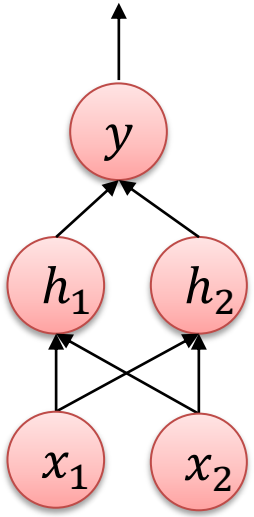

What are $\boldsymbol{h}^{(n)}$ ?

What are $y^{(n)}$?

# XOR problem revisited

- Boolean function:

| $x_1$ | $x_2$ | $t$ | $y$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |

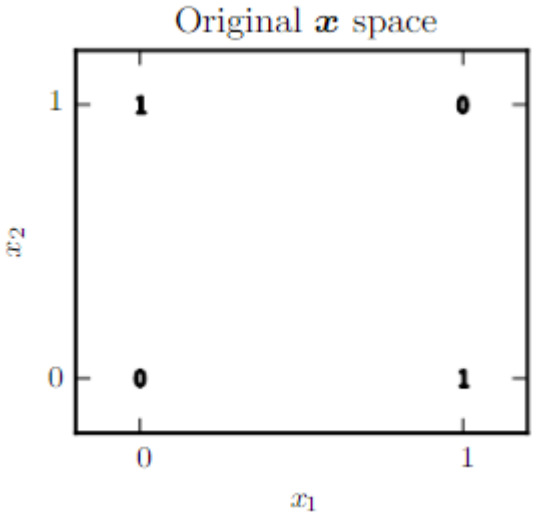


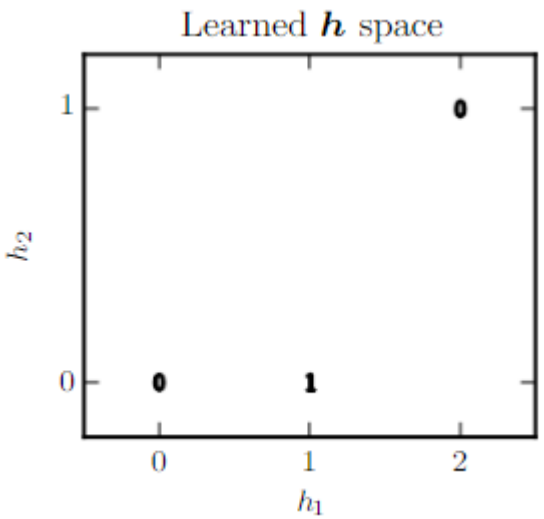

$$y = \text{ReLU}(\boldsymbol{w}^\top \boldsymbol{h} + b)$$

$$\boldsymbol{w} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

$$\boldsymbol{h} = \text{ReLU}(\boldsymbol{V}\boldsymbol{x} + \boldsymbol{c})$$

$$\boldsymbol{V} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \boldsymbol{c} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

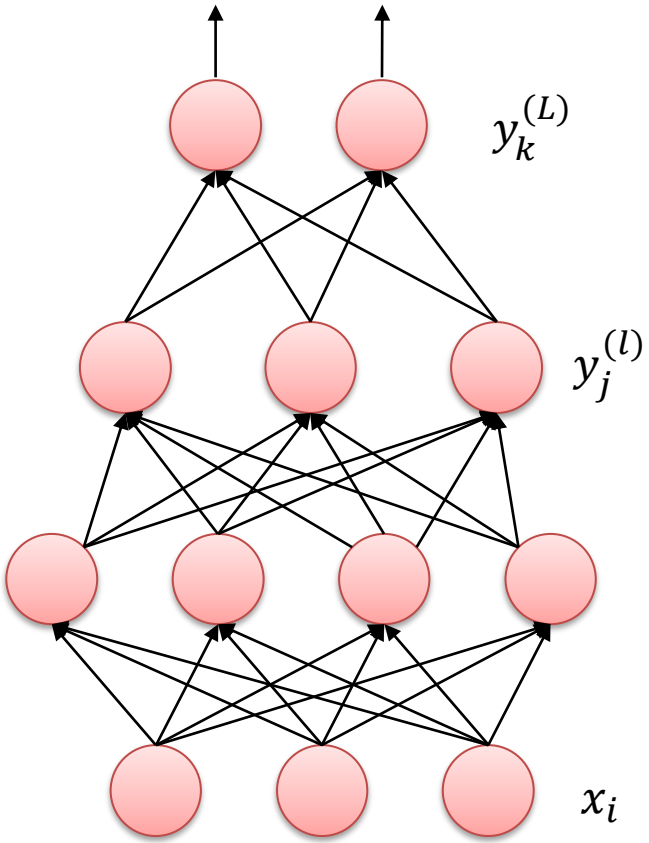



What are $\boldsymbol{h}^{(n)}$ ?

What are $y^{(n)}$?

# Question

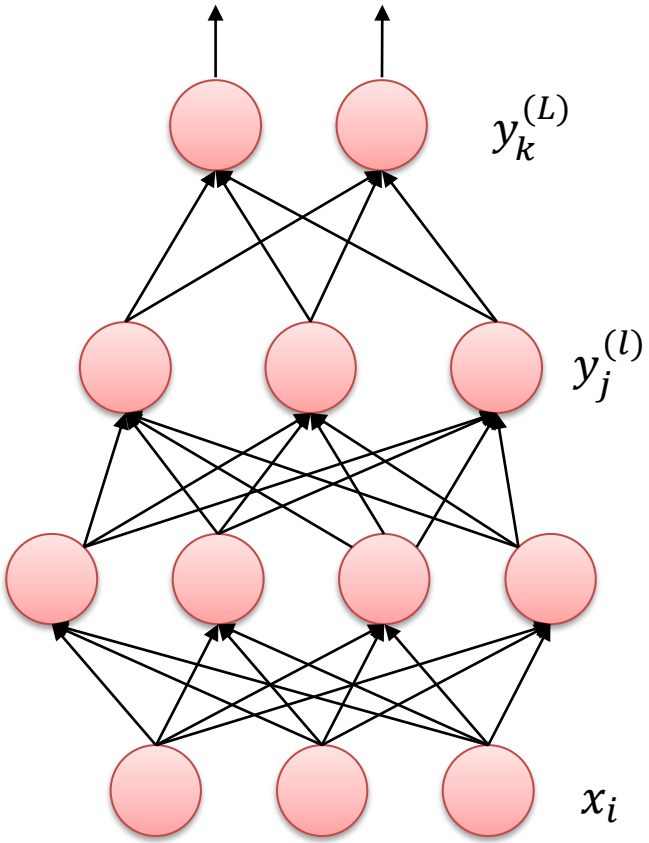



With hidden layers, how to learn the parameters?

Perceptron learning

$$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} + \eta\left(t^{(j)} - y^{(j)}\right)\boldsymbol{x}^{(j)}$$

# Question




With hidden layers, how to learn the parameters?

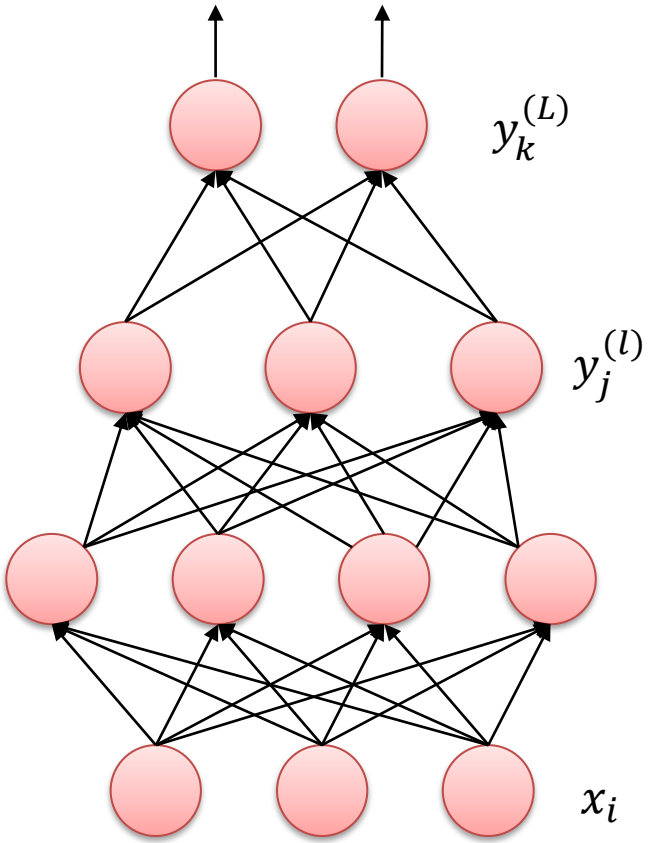
Perceptron learning

$$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} + \eta\left(t^{(j)} - y^{(j)}\right)\boldsymbol{x}^{(j)}$$

Vector addition ➡ Gradient descent

Step function ➡ Continuous fun

# Backpropagation (BP) in history




Different researchers developed the BP algorithm independently



Paul Werbos
1947-



David Rumelhart
1942-2011



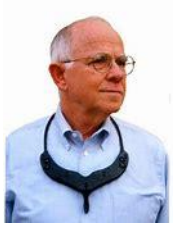
Geoffrey Hinton
1947-



Ronald Williams
?-



Yann LeCun
1960-



Bernard Widrow
1921-

# Loss function

- Loss function

$$E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}$$

where $E^{(n)}$ is the loss function for each input sample $n$

$E^{(n)}$ is a function of $\boldsymbol{y}^{(L)}$

- What form does $E^{(n)}$ take?
  - Regression
  - Classification

多选题 1分

Setting

If the task is regression, $E^{(n)}$ can take

A Cross-entropy loss

B Squared error (L2 loss or Euclidean loss)

C L1 loss

Submit

多选题 1分

Setting

If the task is classification, $E^{(n)}$ can take

A Cross-entropy loss

B Squared error (L2 loss or Euclidean loss)

C L1 loss

Submit

# Loss functions for classification

- Loss function $$E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}$$

where $E^{(n)}$ is the loss function for each input sample $n$

– Squared error, or Euclidean loss

$$E^{(n)} = \frac{1}{2}\sum_{k=1}^{K}(t_k - y_k^{(L)})^2,\ y_k^{(L)} = \frac{1}{1+\exp(-\boldsymbol{w}_k^{(L)\top}\boldsymbol{y}^{(L-1)} - b_k^{(L)})}$$

Is ReLU applicable?

– Cross-entropy loss

$$E^{(n)} = -\sum_{k=1}^{K} t_k \ln y_k^{(L)},\ \ y_k^{(L)} = \frac{\exp(\boldsymbol{w}_k^{(L)\top}\boldsymbol{y}^{(L-1)} + b_k^{(L)})}{\sum_{j=1}^{K}\exp(\boldsymbol{w}_j^{(L)\top}\boldsymbol{y}^{(L-1)} + b_j^{(L)})}$$

where $\boldsymbol{t}$ is target of the form $(0, 0, \ldots, 1, 0, 0)^T$

Except $E^{(n)}$, for clarity, we omit the superscript $(n)$ on $x, t, u, y$ etc. for each input sample.

# Weight adjustment

- Weight adjustment

Learning rate

$$w_{ji}^{(l)} = w_{ji}^{(l)} - \alpha \frac{\partial E}{\partial w_{ji}^{(l)}} \qquad b_j^{(l)} = b_j^{(l)} - \alpha \frac{\partial E}{\partial b_j^{(l)}}$$

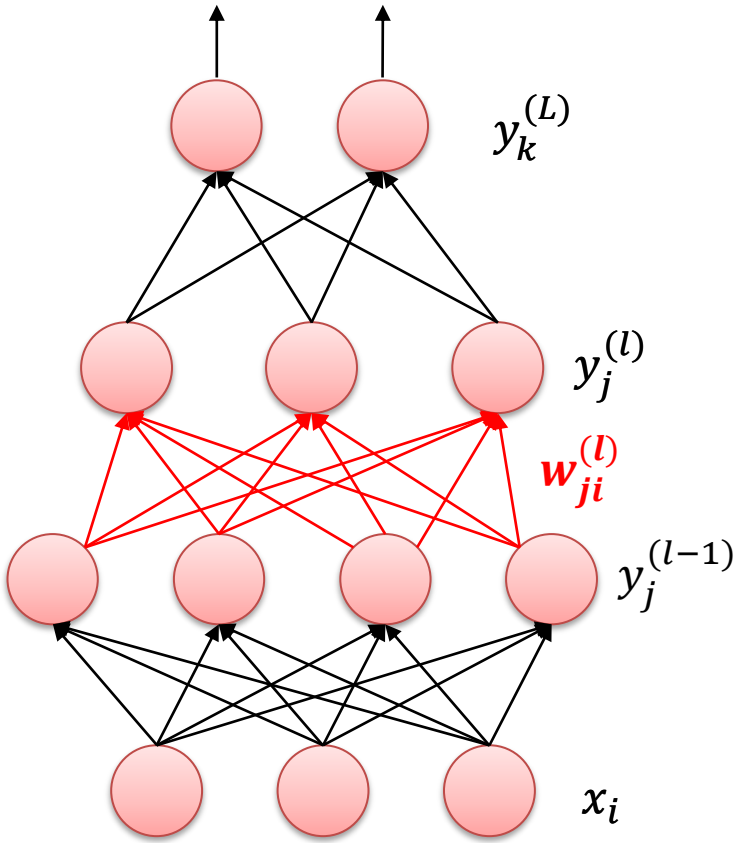
- Weight decay is often used on $w_{ji}^{(l)}$ (not necessary on $b_j^{(l)}$) which amounts to adding an additional term on the cost function

$$J = E + \frac{\lambda}{2} \sum_{i,j,l} (w_{ji}^{(l)})^2$$

- Weight adjustment on $w$ is changed to

$$w_{ji}^{(l)} = w_{ji}^{(l)} - \alpha \frac{\partial J}{\partial w_{ji}^{(l)}} = w_{ji}^{(l)} - \alpha \frac{\partial E}{\partial w_{ji}^{(l)}} - \alpha \lambda w_{ji}^{(l)}$$

# Gradient and local sensitivity




$$u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$$

Hidden layer $\quad y_j^{(l)} = f(u_j^{(l)})$

MSE layer $\quad E^{(n)} = \frac{1}{2}\sum_{j=1}^{K}\left(f\left(u_j^{(L)}\right) - t_j^{(n)}\right)^2$

CE layer $\quad E^{(n)} = -\sum_{j=1}^{K} t_j \ln h(u_j^{(L)})$

All layers have input

$$u^{(l)} = W^{(l)} y^{(l-1)} + b^{(l)}$$

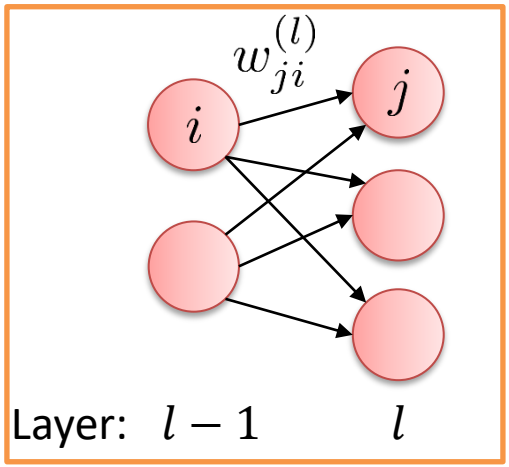
# Gradient and local sensitivity

- Define local sensitivity $\delta_j^{(l)} = \dfrac{\partial E^{(n)}}{\partial u_j^{(l)}}$
- Then for $1 \le l \le L$

$$\frac{\partial E^{(n)}}{\partial w_{ji}^{(l)}} = \frac{\partial E^{(n)}}{\partial u_j^{(l)}} \frac{\partial u_j^{(l)}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} f(u_i^{(l-1)})$$

$$\frac{\partial E^{(n)}}{\partial b_j^{(l)}} = \delta_j^{(l)},$$

since $u_j^{(l)} = \sum_i w_{ji}^{(l)} f(u_i^{(l-1)}) + b_j^{(l)}$, where $f$ is the activation function and $f(u_i^{(0)}) = x_i$



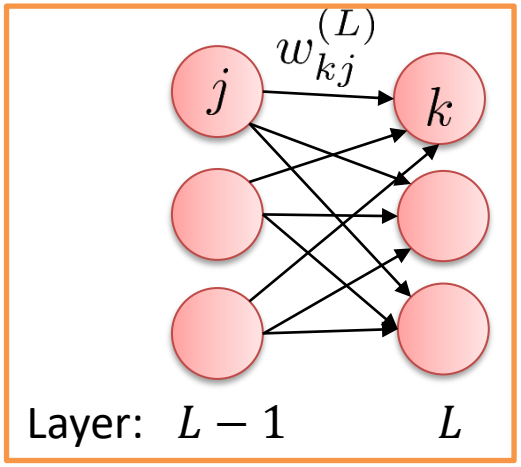

Computing the gradients amounts to computing the local sensitivity in each layer!

# *Recall:* Local sensitivity for MSE layer

- If the squared error is used then the output of the last layer units of MLP are

$$y_k^{(L)} = f(u_k^{(L)}) = f(\boldsymbol{w}_k^{(L)\top} \boldsymbol{y}^{(L-1)} + b_k^{(L)})$$

Output of the units in the (L-1)-th layer




where the activation function $f$ can be

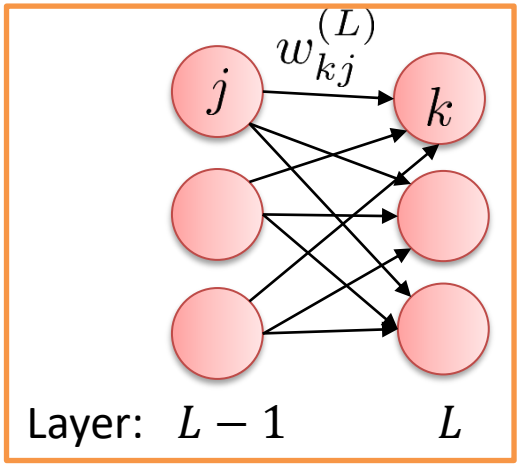
✓ logistic sigmoid ✓ tanh ✓ ReLU

- Recall the error for each sample $E^{(n)} = \frac{1}{2}\sum_{k=1}^{K}(t_k - y_k^{(L)})^2,$
- Local sensitivity

$$\delta_k^{(L)} \triangleq \frac{\partial E^{(n)}}{\partial u_k^{(L)}} = \left(y_k^{(L)} - t_k\right) f'(u_k^{(L)})$$

# *Recall:* local sensitivity for CE layer

- If the softmax regression is used in the last layer of an MLP, the probabilistic function becomes ($\boldsymbol{\theta}$ is replaced with $\boldsymbol{w}^{(L-1)}$ and $b^{(L-1)}$)




Output of the units in the (L-1)-th layer

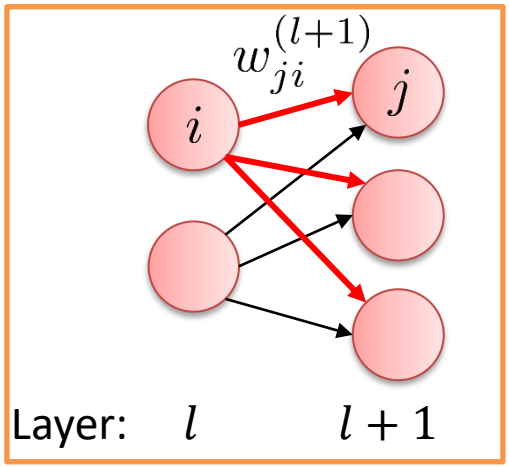
$$y_k^{(L)} \triangleq P(t_k = 1|\boldsymbol{y}^{(L-1)}) = \frac{\exp(\boldsymbol{w}_k^{(L)\top}\boldsymbol{y}^{(L-1)} + b_k^{(L)})}{\sum_{i=1}^{K}\exp(\boldsymbol{w}_i^{(L)\top}\boldsymbol{y}^{(L-1)} + b_i^{(L)})}$$

- Recall the error for each sample $E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$
- Local sensitivity

$$\delta_k^{(L)} \triangleq \frac{\partial E^{(n)}}{\partial u_k^{(L)}} = y_k^{(L)} - t_k$$

# Local sensitivity for other layers

- If $1 \le l < L$, i.e., neuron $i$ is a hidden neuron, it has an effect on all neurons in the next layer, therefore its local sensitivity is




$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}} = \sum_j \frac{\partial E^{(n)}}{\partial u_j^{(l+1)}} \frac{\partial u_j^{(l+1)}}{\partial y_i^{(l)}} \frac{\partial y_i^{(l)}}{\partial u_i^{(l)}} = \sum_j \delta_j^{(l+1)} w_{ji}^{(l+1)} f'(u_i^{(l)})$$

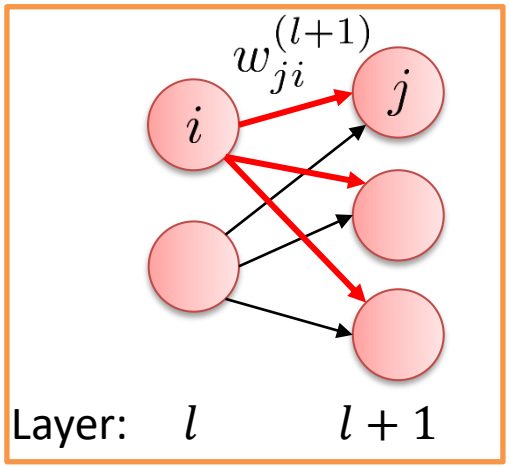
$$u_j^{(l+1)} = \sum_i w_{ji}^{(l+1)} y_i^{(l)} + b_j^{(l+1)} \qquad y_i^{(l)} = f(u_i^{(l)})$$

where $f$ can be any activation function

We compute $\delta_i^{(l)}$ backward, from $l = L, L-1, \dots, 1$, and in the sequel $\partial E / \partial W^{(l)}$ and $\partial E / \partial b^{(l)}$ backward

# Local sensitivity for other layers

- If $1 \leq l < L$, i.e., neuron $i$ is a hidden neuron, it has an effect on all neurons in the next layer, therefore its local sensitivity is




$$\delta_i^{(l)} = \frac{\partial E^{(n)}}{\partial u_i^{(l)}} = \sum_j \frac{\partial E^{(n)}}{\partial u_j^{(l+1)}} \frac{\partial u_j^{(l+1)}}{\partial y_i^{(l)}} \frac{\partial y_i^{(l)}}{\partial u_i^{(l)}} = \sum_j \delta_j^{(l+1)} w_{ji}^{(l+1)} f'(u_i^{(l)})$$

In practice $\boldsymbol{\delta}^{(l)}$ should be calculated in layer $l+1$ because it is determined by the params of layer $l+1$

# Backpropagation in vector-matrix form

- Local sensitivity $\boldsymbol{\delta}^{(l)} = \left( \frac{\partial E^{(n)}}{\partial u_1^{(l)}}, \frac{\partial E^{(n)}}{\partial u_2^{(l)}}, \dots \right)^T$
- For the output layer $L$

MSE: $\boldsymbol{\delta}^{(L)} = (\boldsymbol{y}^{(L)} - \boldsymbol{t}) \odot \boldsymbol{f}'(\boldsymbol{u}^{(L)})$ CE: $\boldsymbol{\delta}^{(L)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$

where $\odot$ denotes element-wise multiplication

- For the hidden layer $1 \leq l < L$

$$\boldsymbol{\delta}^{(l)} = (\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot \boldsymbol{f}'(\boldsymbol{u}^{(l)})$$

- Calculate the gradients $1 \leq l \leq L$

Same dim as $\boldsymbol{W}$

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\boldsymbol{f}(\boldsymbol{u}^{(l-1)}))^\top, \quad \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

for each sample $n$

- Update weights

avg over $n$

$$\boldsymbol{W}^{(l)} = \boldsymbol{W}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} - \alpha\lambda \boldsymbol{W}^{(l)}, \quad \boldsymbol{b}^{(l)} = \boldsymbol{b}^{(l)} - \frac{\alpha}{N} \sum_n \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}}$$

# Gradient vanishing

- Note that for the hidden layer $1 \leq l < L$

$$\boldsymbol{\delta}^{(l)} = (\boldsymbol{W}^{(l+1)})^{\top}\boldsymbol{\delta}^{(l+1)} \odot \boldsymbol{f}'(\boldsymbol{u}^{(l)})$$

  - For logistic function

$$f(z) = \frac{1}{1+\exp(-z)}$$

    we have $f'(z) = f(z)\big(1 - f(z)\big) < 1$

  - For the tanh function

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

    we have $f'(z) = 1 - \tanh^2(z) < 1$

How about ReLU function?

- For these two sigmoid functions, $\boldsymbol{\delta}^{(l)}$ is smaller and smaller from $L$ to 1. The gradient approaches zero in lower layers

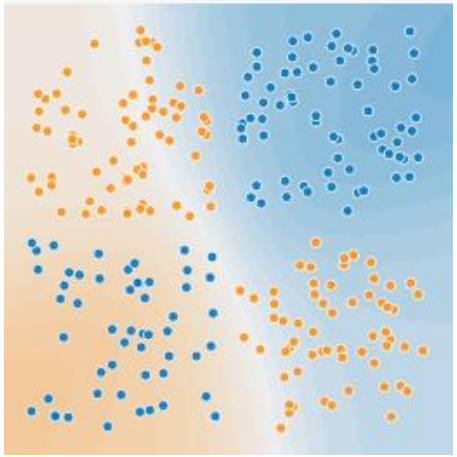
$$\frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\boldsymbol{f}(\boldsymbol{u}^{(l-1)}))^{\top}, \quad \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

# Implementation

- Run forward process
  - Calculate $f(u^l)$ and $f'(u^l)$ for $l = 1,2,\dots,L$
- Run backward process
  - Calculate $\delta^{(l)}$ and $\partial E/\partial W^{(l)}$,$\partial E/\partial b^{(l)}$ for $l = L, L-1,\dots,1$
- Update $W^{(l)}$ and $b^{(l)}$ for $l = 1,2,\dots,L$
- Implementation
  - Implement the layer as a class and provide functions for forward calculation and backward calculation, respectively
  - The forward functions and backward functions differ according to the type of the layer, e.g., input layer, hidden layer, softmax output layer, sigmoid output layer, etc.
  - Then you can design different structures of MLP by specifying the layer modules in a main file

# Experiment: Classification of 2D points

- http://playground.tensorflow.org



Network setting

- Input: x1, x2
- Hidden layer: 1 layer, 4 neurons
- Use default values for other hyper-parameters

1. Run the training process
2. Change the learning rate to 1 and run
3. Change the learning rate back to 0.03, but use a regularization “L2” with rate 0.01, 0.1, or 1
4. Add a 2nd hidden layer with 2 neurons, and run

# Experiment: Classification of 2D points

- http://playground.tensorflow.org



Set a suitable setting for solving this problem

- Change the hidden layers, input representation, learning rates, regularization…

# Summary of Part 1

**Forward calculation**

For $l = 1, \ldots, L$: $\boldsymbol{u}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)} + \boldsymbol{b}^{(l)}$ and $\boldsymbol{y}^{(l)} = \boldsymbol{f}(\boldsymbol{u}^{(l)})$

**Backward calculation**

For $l = L$: $\boldsymbol{\delta}^{(L)} = (\boldsymbol{y}^{(L)} - \boldsymbol{t}) \odot \boldsymbol{f}'(\boldsymbol{u}^{(L)})$ or $\boldsymbol{\delta}^{(L)} = \boldsymbol{y}^{(L)} - \boldsymbol{t}$

For $l = L - 1, \ldots, 1$: $\boldsymbol{\delta}^{(l)} = (\boldsymbol{W}^{(l+1)})^{\top}\boldsymbol{\delta}^{(l+1)} \odot \boldsymbol{f}'(\boldsymbol{u}^{(l)})$

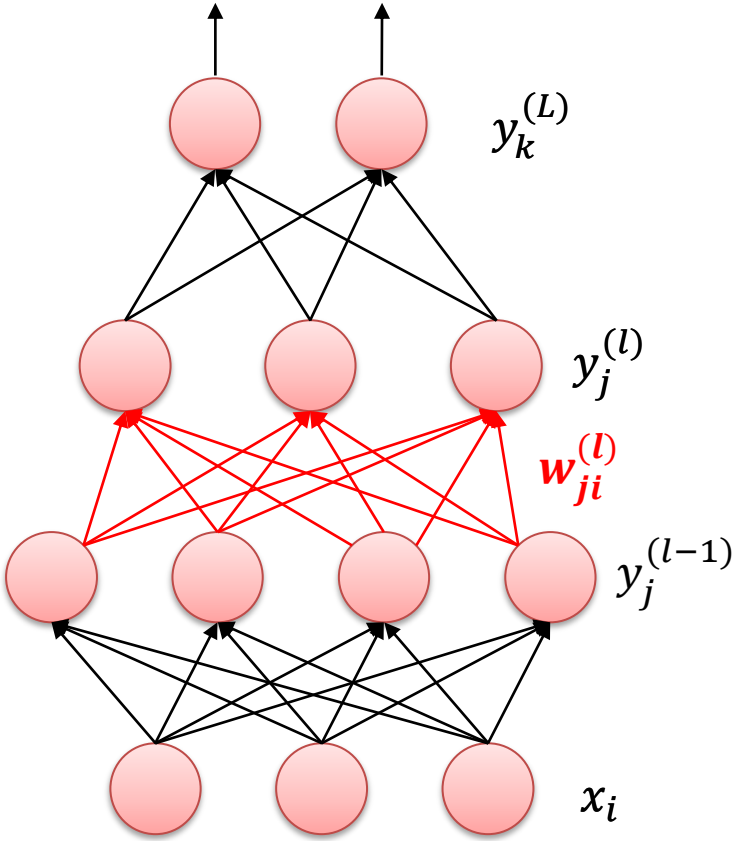
$$\frac{\partial E^{(n)}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\boldsymbol{f}(\boldsymbol{u}^{(l-1)}))^{\top}, \quad \frac{\partial E^{(n)}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

**Gradient vanishing**

# Outline

I. Multi-layer perceptron
II. Layer decomposition
III. Training techniques-I
IV. Summary

# Motivation




$$u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$$

Hidden layer $y_j^{(l)} = f(u_j^{(l)})$ Sigmoid/tanh /ReLU

MSE layer $E^{(n)} = \frac{1}{2}\sum_{j=1}^{K}\left(f\left(u_j^{(L)}\right) - t_j^{(n)}\right)^2$

CE layer $E^{(n)} = -\sum_{j=1}^{K} t_j \ln h(u_j^{(L)})$

All layers have input

$$u^{(l)} = W^{(l)} y^{(l-1)} + b^{(l)}$$

The implementation is tedious!

# Layer decomposition

- The input layer or hidden layer

$$y_j^{(l)} = f\left(\sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}\right)$$

can be decomposed into two layers

  - Fully connected layer: $u_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$
  - Activation layer: $y_j^{(l)} = f(u_j^{(l)})$

In every new layer, you can derive the local sensitivity and the gradient of the parameters (if any)

# Layer decomposition

- The squared error layer $E^{(n)} = \frac{1}{2}\left|\left|\boldsymbol{f}\left(\boldsymbol{u}^{(L)}\right) - \boldsymbol{t}\right|\right|^2$ can be decomposed into two layers
  - Activation layer: $y_k^{(L)} = f(u_k^{(L)})$, where $f$ can be any function
  - Loss layer: $E^{(n)} = \frac{1}{2}\left|\left|\boldsymbol{y}^{(L)} - \boldsymbol{t}\right|\right|^2$

In every new layer, you can derive the local sensitivity and the gradient of the parameters (if any)

单选题 1分

设置

Consider the squared error function

$$E^{(n)} = \frac{1}{2}\left\|\boldsymbol{f}\left(\boldsymbol{W}^{(L)}\boldsymbol{y}^{(L-1)} + \boldsymbol{b}^{(L)}\right) - \boldsymbol{t}\right\|^2$$

How many layers shall be designed?

A 1

B 2

C 3

D 4

提交

# Question

- Consider the squared error function

$$E^{(n)} = \frac{1}{2}\left\|\boldsymbol{f}\left(\boldsymbol{W}^{(L)}\boldsymbol{y}^{(L-1)} + \boldsymbol{b}^{(L)}\right) - \boldsymbol{t}\right\|^2$$

How many layers shall be designed?

FC layer + activation layer + Euclidean loss layer

# Layer decomposition

- The Cross-entropy loss layer $E^{(n)} = -\sum_{k=1}^{K} t_k \ln f\left(u_k^{(L)}\right)$ can be decomposed into two layers
  - Softmax layer: $y_k^{(L)} = f(u_k^{(L)})$, where $f$ is the softmax function
  - Loss layer: $E^{(n)} = -\sum_{i=1}^{K} t_i \ln y_i^{(L)}$

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{u}_k^{(L)}} = \sum_{i=1}^{K} \frac{\partial E^{(n)}}{\partial y_i^{(L)}} \frac{\partial y_i^{(L)}}{\partial u_k^{(L)}} = \sum_{i=1}^{K} \left(-t_i \frac{1}{y_i^{(L)}}\right) \left(y_i^{(L)}(\Delta_{i,k} - y_k^{(L)})\right)$$

Doing this is unnecessary! Why?

$$= -\left(\sum_{i=1}^{K} t_i \Delta_{i,k} - \sum_{i=1}^{K} t_i y_k^{(L)}\right) = -\left(t_k - y_k^{(L)}\right)$$

# 单选题 1分

设置

Consider the CE loss function

$$E^{(n)} = -\sum_{k=1}^{K} t_k \ln f\left(\sum_i w_{ki}^{(l)} y_i^{(l-1)} + b_k^{(l)}\right)$$

How many layers shall be designed?

A 1

B 2

C 3

D 4

提交

# Question

- Consider the CE loss function

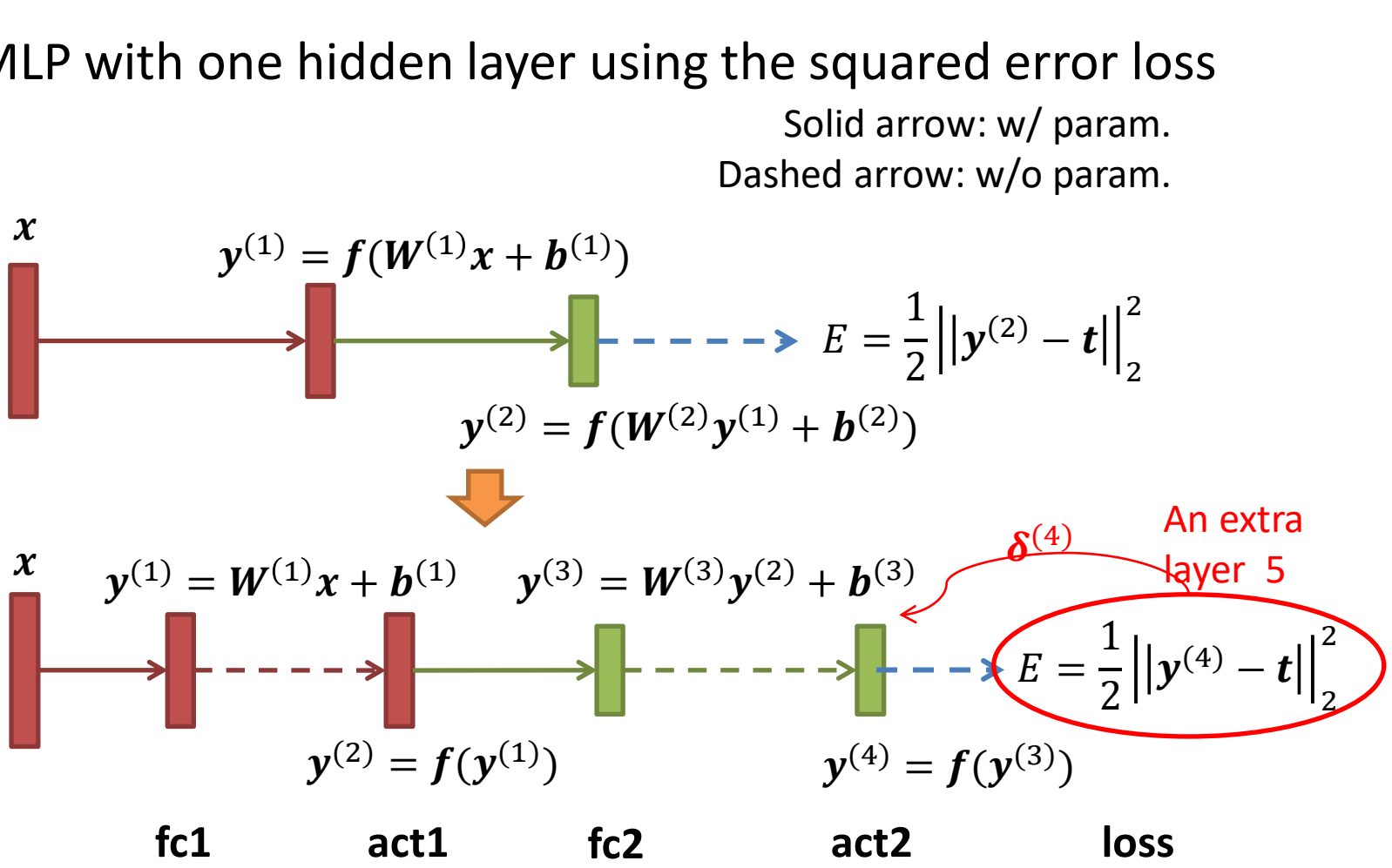
$$E^{(n)} = -\sum_{k=1}^{K} t_k \ln f\left(\sum_i w_{ki}^{(l)} y_i^{(l-1)} + b_k^{(l)}\right)$$

How many layers shall be designed?

FC layer + softmax cross-entropy layer

# Example 1

- An MLP with one hidden layer using the squared error loss

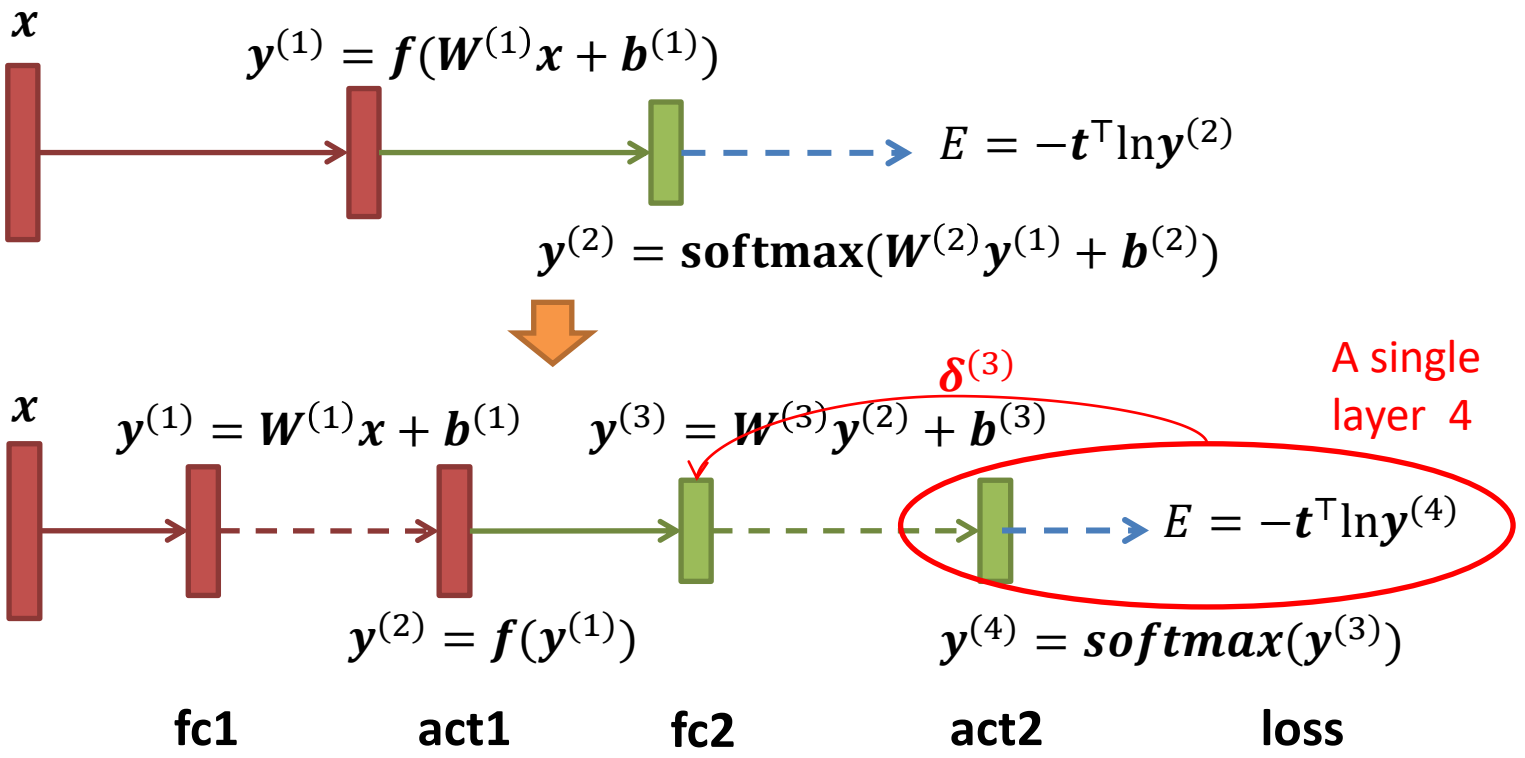



$\boldsymbol{u}^{(l)}$ and $\boldsymbol{y}^{(l)}$ are the same in every layer $l$

# Example 2

- An MLP with one hidden layer using the Cross-entropy loss

Solid arrow: w/ param.
Dashed arrow: w/o param.




$\boldsymbol{u}^{(l)}$ and $\boldsymbol{y}^{(l)}$ are the same in every layer $l$

# Backward calculation (Exercise)

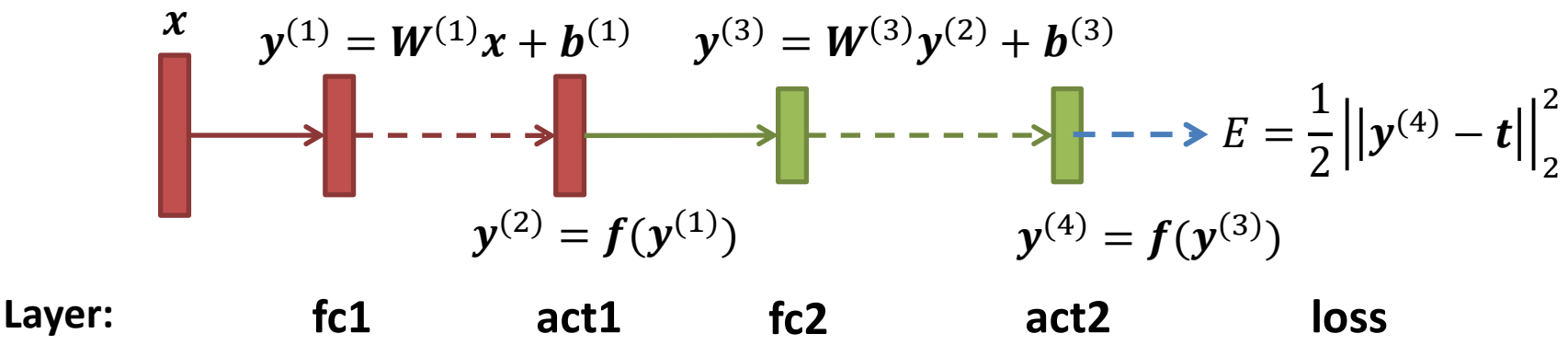
- Derive the local sensitivity $\delta$ and gradient $\partial E/\partial w$ and $\partial E/\partial b$ where applicable for
  - Euclidean loss layer: $E^{(n)} = \frac{1}{2}\left\|\boldsymbol{y}^{(L)} - \boldsymbol{t}\right\|^2$ ← $L+1$ layers in total
    - Note that here we calculate $\boldsymbol{\delta}^{(L)} = \partial E^{(n)}/\partial \boldsymbol{y}^{(L)}$
  - Softmax-Cross-entropy loss layer $E^{(n)} = -\sum_{k=1}^{K} t_k \ln f\left(y_k^{(L)}\right)$ ← $L$ layers in total
    - Note that here we calculate $\boldsymbol{\delta}^{(L-1)} = \partial E^{(n)}/\partial \boldsymbol{y}^{(L-1)}$
  - Fully connected layer: $y_j^{(l)} = \sum_i w_{ji}^{(l)} y_i^{(l-1)} + b_j^{(l)}$
  - Sigmoid layer: $y_j^{(l)} = f(y_j^{(l-1)})$, where $f$ is a sigmoid function
  - ReLU layer: $y_j^{(l)} = f(y_j^{(l-1)})$, where $f$ is a ReLU function

Note: $\boldsymbol{u}^{(l)}$ and $\boldsymbol{y}^{(l)}$ are identical in every layer $l$

Watch the supplementary video in WebLearning for hints

# Summary of Part 2

- All layers discussed so far can be decomposed
  - Input layer, hidden layer, logistic regression layer, softmax regression layer, loss layer




One exception: softmax function and cross-entropy loss function are bound together

# Outline

I. Multi-layer perceptron
II. Layer decomposition
III. Training techniques-I
IV. Summary

# (1) Weight initialization

$W$ inputting to a neuron is drawn from a distribution:

- Gaussian
  - a Gaussian distribution with zero mean and fixed std, e.g., 0.01
- Xavier
  - a distribution with zero mean and a specific std $1/\sqrt{n_{\text{in}}}$ where $n_{\text{in}}$ is the number of neurons feeding into the neuron
  - Gaussian distribution or uniform distribution is often used
- MSRA
  - a Gaussian distribution with zero mean and a specific std $2/\sqrt{n_{\text{in}}}$

# (2) Learning rate

- In SGD the learning rate $\alpha$ is typically much smaller than a corresponding learning rate in batch gradient descent because there is much more variance in the update.
- Choosing the proper schedule
  - Use a small enough constant learning rate that gives stable convergence in the initial epoch (full pass through the training set) or two of training and then halve the value of the learning rate as convergence slows down
  - Evaluate a held out set after each epoch and anneal the learning rate when the change in objective between epochs is below a small threshold
  - Anneal the learning rate at each iteration $t$ as $\frac{a}{b+t}$ where $a$ and $b$ dictate the initial learning rate and when the annealing begins respectively

# (3) Order of training samples

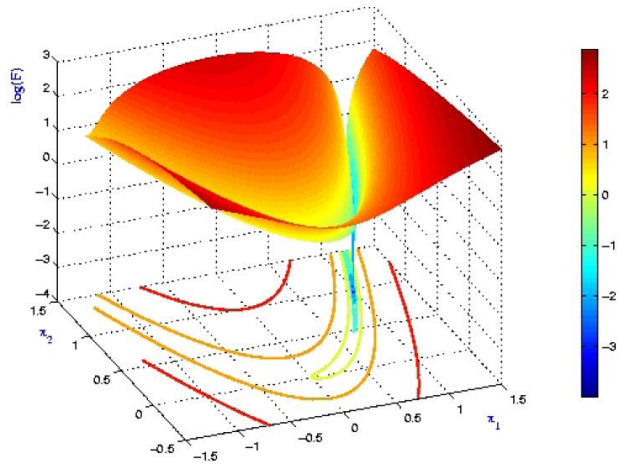
- If the data is given in some meaningful order, this can bias the gradient and lead to poor convergence
- Generally a good method to avoid this is to randomly shuffle the data prior to each epoch of training.

# Pathological curvature

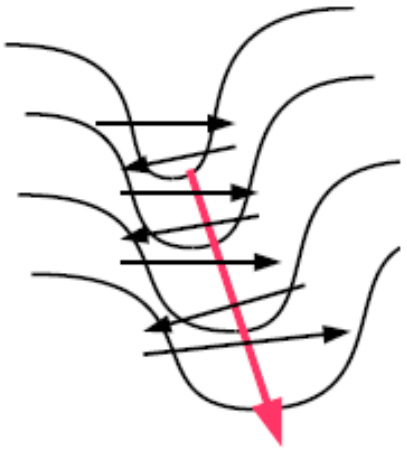
- The objective has the form of a long shallow ravine leading to the optimum and steep walls on the sides

Rosenbrock function

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$




- The objectives of deep architectures have this form near local optima and thus standard SGD tends to oscillate across the narrow ravine



Black arrows: gradient descent paths

# (4) Momentum

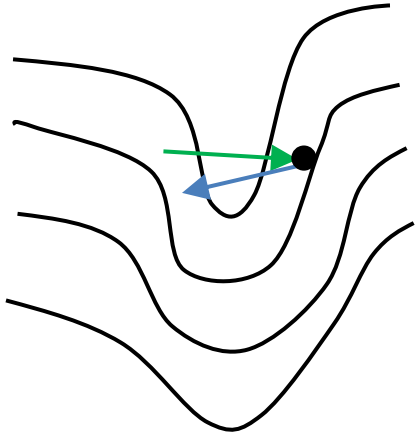
- Momentum is one method for pushing the objective more quickly along the shallow ravine
- The momentum update is given by,

$$\boldsymbol{v} = \gamma \boldsymbol{v} - \alpha \nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} + \boldsymbol{v}$$

  - $\boldsymbol{v}$ is the current velocity vector, which accumulates the previous gradients
  - $\gamma \in (0,1]$ determines for how many iterations the previous gradients are incorporated into the current update
  - One strategy: $\gamma$ is set to 0.5 until the initial learning stabilizes and then is increased to 0.9 or higher

# An example

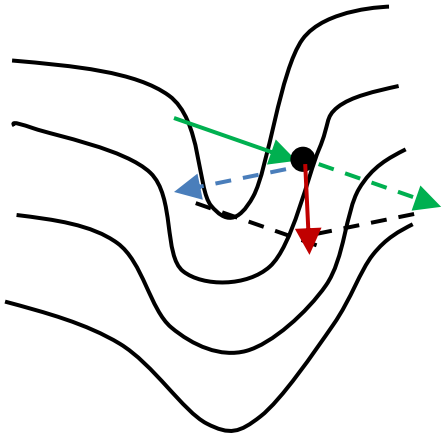


Let $\boldsymbol{g} = -\nabla_{\boldsymbol{\theta}} J\left(\boldsymbol{\theta}; \boldsymbol{x}^{(i)}, \boldsymbol{t}^{(i)}\right)$

Green: change of $\theta$ in the previous step

- Standard gradient decent:

$$\Delta\boldsymbol{\theta} = \alpha\boldsymbol{g}$$



- Gradient decent with momentum:

$$\Delta\boldsymbol{\theta} = \gamma\boldsymbol{v} + \alpha\boldsymbol{g}$$

This $\Delta\boldsymbol{\theta}$ is better aligned with the decreasing direction of the ravine

# Summary of Part 3

- Weight initialization
- Learning rate
- Randomly shuffle
- Momentum

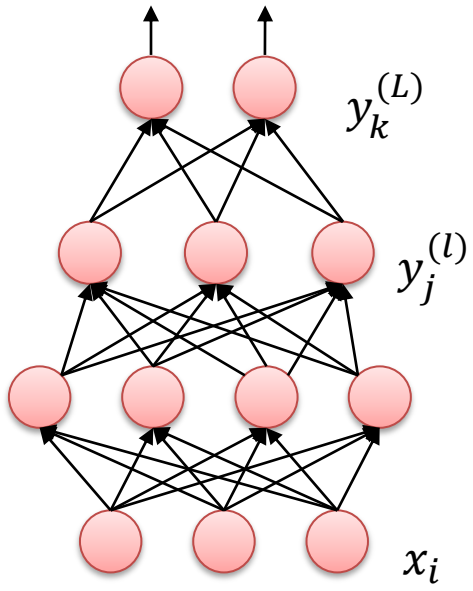
# Outline

I. Multi-layer perceptron
II. Layer decomposition
III. Training techniques-I
IV. Summary

# Summary of this lecture

## knowledge

### I. Multi-layer perceptron




- Forward calculation
- Backward calculation

### II. Layer decomposition

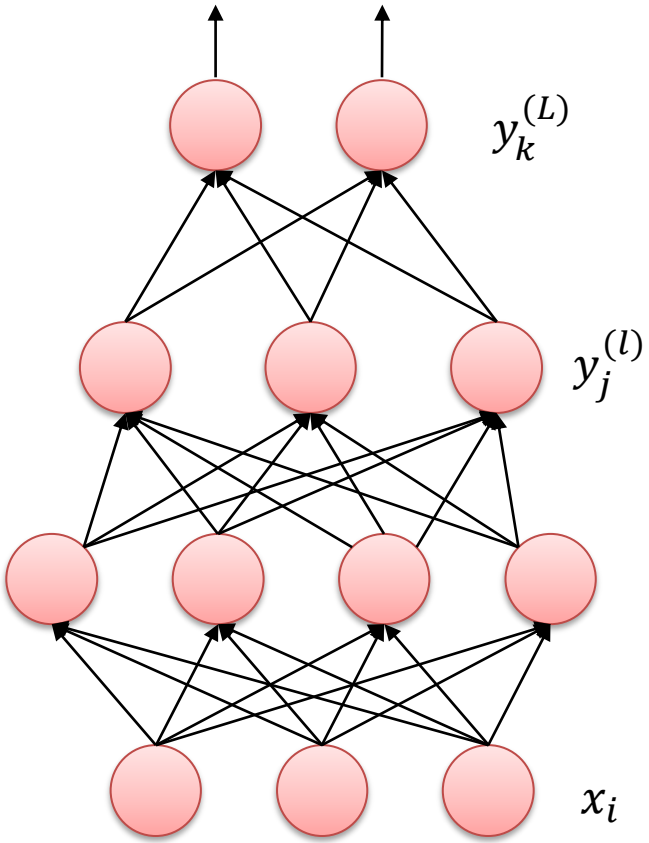
- FC layer
- ReLU layer
- sigmoid layer
- tanh layer
- Euclidian loss layer
- Softmax CE loss layer

### III. Training techniques-I

- Weight initialization
- Learning rate
- Momentum
- Randomly shuffle

# Summary of this lecture

## Capability and value






Paul Werbos
1947-



David Rumelhart
1942-2011



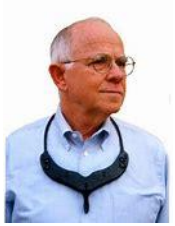
Geoffrey Hinton
1947-



Ronald Williams
?-



Yann LeCun
1960-

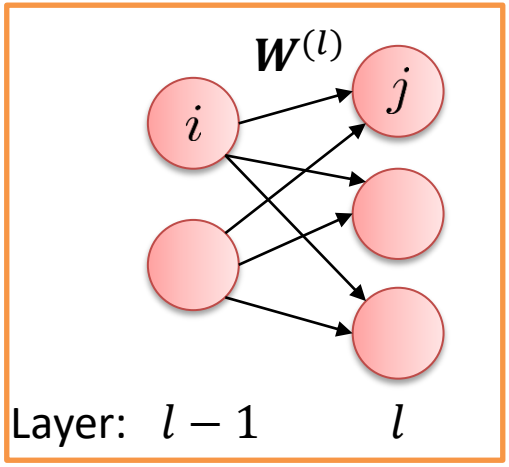


Bernard Widrow
1921-

**Useful research will be recognized!**

# Prepare for the next lecture

- Form groups of two and every group chooses one of the following papers for reading
- Share with us the main conclusion and your opinion on it.
  - Smith, Cyclical learning rates for training neural networks, IEEE Winter Conference on Applications of Computer Vision, 2017
  - Liao, Leibo, Poggio, How important is weight symmetry in backpropagation? AAAI, 2016




$$\boldsymbol{u}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)} + \boldsymbol{b}^{(l)}$$

$$\boldsymbol{\delta}^{(l-1)} = \left(\boldsymbol{W}^{(l)}\right)^{\mathsf{T}}\boldsymbol{\delta}^{(l)}$$

# Video material

- Watch the supplementary video uploaded to WebLearning
  - Hint for calculating the local sensitivity and gradient in decomposed layers

# Homework X2

- MLP for MNIST classification
  - Deadline: Oct 27 (Monday noon) 12:00